

1. malloc() is declared in which header file?

- stdio.h
- ✓ **stdlib.h**
- string.h
- malloc.h

Explanation: The standard memory allocation functions like malloc(), calloc(), realloc(), and free() are defined in the C standard library header <stdlib.h> (or <cstdlib> in C++). While some older compilers support a non-standard <malloc.h>, <stdlib.h> is the standard-compliant, portable choice.

2. Which function initializes all allocated bytes to zero?

- malloc()
- ✓ **calloc()**
- realloc()
- new

Explanation: calloc() (contiguous allocation) allocates a specified amount of memory and automatically initializes every bit of that memory area to zero. In contrast, malloc() leaves the memory uninitialized, containing whatever random "garbage values" were previously left there.

3. Which function can increase or decrease the size of previously allocated memory?

- calloc()
- ✓ **realloc()**
- free()
- new

Explanation: The realloc() function updates the size of a previously allocated dynamic memory block. It can expand or contract the memory. If necessary, it handles moving the data to a new memory block entirely and automatically releases the old block.

C++ Memory Management

4. Which operator is used for dynamic memory allocation in C++?

- malloc
- alloc
- ✓ **new**
- create

Explanation: C++ introduced the native new operator to allocate memory dynamically on the heap. Unlike C-style functions like malloc(), new calculates the required object size automatically and invokes object constructors.

5. Memory allocated using new resides in

- Stack
- ✓ **Heap**
- Cache
- Register

Explanation: All objects created dynamically via new (or C functions like malloc) are allocated out of the "Heap" (or free store). The heap is an unorganized pool of memory used for variables that need to persist beyond the scope of the function that created them.

6. Local variables are usually stored in

- Heap
- ✓ **Stack**
- ROM
- Global Memory

Explanation: Local variables defined inside a function block are managed via automatic storage duration. They are allocated in the function's activation record on the **Stack** memory structure, which grows and shrinks automatically as functions are entered and exited.

Memory Deallocation and Safety

7. Which statement correctly deallocates an array allocated using new?

- delete arr;
- ✓ **delete[] arr;**
- free(arr);
- remove(arr);

Explanation: When you allocate an array using new Type[size], you must use the array version of the delete operator: delete[] arr;. This tells the runtime system to look up the array size and properly execute the destructors for *all* elements in that array.

8. Which statement is true regarding malloc()?

- Calls constructor
- Initializes memory
- ✓ **Returns void***
- Calls destructor

Explanation: malloc() is a pure C library function. It has no understanding of C++ classes, so it cannot call constructors or destructors. It does not clear memory to zero; it simply requests

bytes and returns a raw type-agnostic generic pointer: void*.

9. Which statement is true regarding new?

- Returns void*
- ✓ **Calls constructor**
- Cannot allocate arrays
- Exists only in C

Explanation: The C++ new operator operates at a higher abstraction layer than raw byte allocation. It calculates size based on the type, allocates the bytes, and then explicitly executes the object's constructor to initialize the instance. It returns a fully typed pointer.

Memory Problems and Pointer Types

10. Memory leak occurs when

- Memory is freed twice
- ✓ **Allocated memory is never released**
- Pointer becomes NULL
- Stack becomes full

Explanation: A memory leak happens when a program dynamically provisions heap memory but loses track of its reference pointers without calling free() or delete. This leaves the memory claimed but inaccessible, steadily depleting the operating system's resources.

11. Which pointer refers to already deallocated memory?

- Null pointer
- ✓ **Dangling pointer**
- Wild pointer
- Void pointer

Explanation: A **dangling pointer** is a pointer that continues to point to a specific memory address even after that underlying block of memory has been deallocated or freed. Attempting to read or write through a dangling pointer leads to memory corruption or crashes.

12. Which pointer is never initialized?

- Null pointer
- Dangling pointer
- ✓ **Wild pointer**
- Smart pointer

Explanation: A **wild pointer** is created when a pointer variable is declared but not initialized to any valid address or nullptr. It retains whatever random memory addresses were left over on the stack frame, making dereferencing it extremely dangerous.

Modern C++ Features

13. Which is true about nullptr in C++?

- ✓ **It is a type-safe null pointer**
- It is same as malloc
- It stores heap address
- It deletes memory

Explanation: Introduced in C++11, nullptr is a literal representing a type-safe null pointer of type std::nullptr_t. Unlike the traditional macro NULL (which resolves simply to integer 0), nullptr cannot be implicitly converted to structural numeric types, preventing method overloading ambiguity.

Stack vs Heap Performance

14. Which memory allocation is generally faster?

- Heap allocation
- ✓ **Stack allocation**
- Both always equal
- Depends only on RAM size

Explanation: Stack allocation is exceptionally fast because it only requires updating a single CPU register—the Stack Pointer (adding or subtracting a value). Heap allocation requires searching through complex tables or free lists for free memory segments, incurring operating system overhead.

15. Which memory is automatically reclaimed after a function returns?

- Heap
- ✓ **Stack**
- ROM
- Hard disk

Explanation: When a function returns, its whole stack frame (activation record) is popped off the CPU **Stack**. All local variables defined inside that scope vanish instantly, reclaiming memory overhead without any explicit instructions from the developer.

Memory Management Issues

16. What happens if delete is applied twice to the same pointer?

- Guaranteed correct output
- ✓ **Undefined behavior**
- Compile-time success with no issue
- Memory automatically doubles

Explanation: Executing delete or free() on the same active pointer twice (double-free) corrupts the internal tracking structures of the memory allocator. The ISO C++ standard defines this as **Undefined Behavior**, frequently causing critical crashes or security exploits.

17. Which memory region commonly suffers from fragmentation?

- ✓ **Heap**
- Stack
- CPU register
- ROM

Explanation: Because allocations and deallocations on the **Heap** occur arbitrarily at runtime in any order and in varying sizes, it quickly breaks down into an unorganized patchwork of free and occupied chunks (fragmentation). The Stack is contiguous and does not suffer from this.

18. Which fragmentation occurs because free memory is scattered in small blocks?

- Internal fragmentation
- ✓ **External fragmentation**
- Stack fragmentation
- Cache fragmentation

Explanation: External Fragmentation describes a state where total free memory exists to satisfy an allocation request, but the memory is broken up and scattered into tiny non-contiguous blocks. As a result, a large sequential block request will fail.

19. Which fragmentation occurs due to unused space inside an allocated block?

- External fragmentation
- ✓ **Internal fragmentation**
- Paging error
- Stack overflow

Explanation: Internal Fragmentation occurs when an allocator provisions a block of memory that is slightly larger than requested (often due to cell alignment requirements like 8-byte boundaries). The unused leftover space inside that allocated block is wasted and cannot be reused elsewhere.

STL and Dynamic Memory

20. Which of the following uses dynamic memory internally?

- vector
- string
- list
- ✓ **All of these**

Explanation: Standard Template Library (STL) classes like `std::vector`, `std::string`, and `std::list` are designed to dynamically grow and shrink at runtime. They use an allocator abstraction layer to provision heap memory automatically under the hood.

Stack Behavior

21. Stack follows which allocation strategy?

- FIFO
- ✓ **LIFO**
- Random
- Circular

Explanation: The **Stack** operates on a **LIFO** (Last In, First Out) basis. The function context most recently added to the stack frame is inevitably the first one finished and popped off when execution control returns to the caller.

Heap Characteristics

22. Heap memory generally supports

- Compile-time fixed allocation
- ✓ **Dynamic allocation**
- Only LIFO allocation
- Only global variables

Explanation: The definitive purpose of the heap is supporting explicit **Dynamic Allocation**. It acts as an open-ended workspace for objects whose lifetimes cannot be predicted or bound to structural compilation-time constraints or linear function cycles.

Stack Overflow and Recursion

23. Which situation may cause stack overflow?

- ✓ **Infinite recursion**
- Correct use of delete
- Using cout
- Declaring int x

Explanation: Every recursive call creates a new stack frame. If a recursive routine is infinite or lacks a correct base exit condition, frames accumulate continuously until they exhaust the limited stack memory boundary, triggering a **Stack Overflow** crash.

24. Recursive functions primarily consume

- Heap memory
- ✓ **Stack memory**
- Cache only

- ROM

Explanation: Each active call invocation requires processing overhead storage—local tracking parameters, variable scopes, and return instruction address pointers. This context block is pushed directly onto the execution thread's local **Stack memory**.

Pointer Operations

25. What is printed conceptually by `int *p = new int; cout << p;`?

- Value stored in integer
- ✓ **Address of allocated heap memory**
- Always zero
- Compiler name

Explanation: The variable `p` is declared as a pointer tracking integer coordinates. Passing the raw pointer value `p` directly into `std::cout` prints the actual hexadecimal address of the newly provisioned space out on the heap.

26. What is printed by `int x=10; int *p=&x; cout<<*p;`?

- Address of `x`
- ✓ **10**
- Address of `p`
- Garbage always

Explanation: Here, `p` tracks the address of variable `x`. Prefacing a pointer variable with the asterisk symbol (`*p`) denotes **dereferencing**. It instructs the computer to read the actual value stored at that target address, which is **10**.

27. Which operator returns the address of a variable?

- *
- ✓ **&**
- %

Explanation: The ampersand (`&`) functions as the **address-of operator** when used in an expression. Placing it in front of an initialized variable evaluates directly to its specific memory address context location.

Pointer Operators and Memory Safety

28. Which operator accesses value stored at an address?

- &
- ** ✓ ***
- @
- \$

Explanation: The asterisk symbol (*) is known as the indirection or **dereference operator**. Prefixed onto an address pointer value, it navigates directly to that coordinates block to read or write the actual value held inside.

29. Which is true about delete p;?

- ✓ **It deletes heap memory pointed by p**
- It deletes the pointer variable from stack
- It always sets p to NULL
- It is used for stack variables

Explanation: Executing delete p; releases the heap memory block that p points to back to the system. However, the pointer variable p itself continues to exist on the stack and still retains the now-invalid address until it goes out of scope or is manually cleared.

30. Can free() be safely used on memory allocated using new?

- Yes, always
- ✓ **No, it is undefined behavior**
- Only for int
- Only for arrays

Explanation: Mixing dynamic paradigms is strictly forbidden. free() does not call constructors or destructors and uses an entirely different tracking mechanism than new. Mixing them triggers memory corruption or immediate runtime application faults.

Memory Allocation Compatibility

31. Can delete be safely used on memory allocated using malloc()?

- Yes, always
- ✓ **No, it is undefined behavior**
- Only for char
- Only for void*

Explanation: Just like using free() on memory allocated with new, using delete on memory allocated via malloc() is undefined behavior. You must always match malloc with free, and new with delete.

Allocation Types

32. Which memory allocation happens at runtime?

- Static allocation
- ✓ **Dynamic allocation**
- Compile-time allocation
- Preprocessor allocation

Explanation: Dynamic Allocation occurs explicitly while the application is running. It allows programs to request and size memory blocks adaptively based on live runtime user actions and input data.

33. Which memory allocation size is usually fixed before program execution?

- Dynamic allocation
- ✓ **Static allocation**
- Heap allocation only
- Runtime allocation

Explanation: Static allocation (and automatic allocation on the stack) requires sizes to be explicitly determined at compile-time. Once the binary is compiled, these allocation boundaries are fixed and cannot expand during execution.

Dynamic Memory Requirements

34. Which of the following typically requires a pointer to access allocated memory?

- malloc()
- calloc()
- new
- ✓ **All of these**

Explanation: Because dynamic allocation constructs variables anonymously within the remote heap space without an explicit named identifier, all mechanics (malloc, calloc, and new) return the address of the memory block, requiring a pointer to access it.

Language Features

35. Which language introduced new/delete operators for object allocation?

- C
- ✓ **C++**
- Assembly
- SQL

Explanation: Bjarne Stroustrup introduced the type-safe new and delete operators in **C++** to support object-oriented programming, ensuring constructors and destructors are automatically called.

Binding Types

36. Static binding is also known as

- ✓ **Early binding**
- Late binding
- Dynamic binding

- Runtime binding

Explanation: Static binding is also referred to as **Early binding** because the compiler maps function calls to specific definitions at compile-time, well before the application binary executes.

37. Dynamic binding is also known as

- Compile-time binding
- Early binding
- ✓ **Late binding**
- Static binding

Explanation: Dynamic binding is known as **Late binding** because the decision of which specific function implementation to run is deferred until runtime, based on the actual type of the polymorphic object.

38. Virtual functions in C++ use

- Early binding
- ✓ **Late binding**
- Static binding
- Preprocessor binding

Explanation: In C++, declaring a function as virtual tells the compiler to use **Late binding**. The compiler generates a virtual table (vtable) to look up the correct function at runtime based on the actual object type.

Polymorphism

39. Function overloading is an example of

- ✓ **Compile-time polymorphism**
- Runtime polymorphism
- Memory leak
- Fragmentation

Explanation: Function overloading allows multiple functions with the same name but different signatures. The compiler resolves which function to call at compile-time, making it an example of **Compile-time polymorphism**.

40. Function overriding with virtual functions is an example of

- Compile-time binding
- ✓ **Runtime polymorphism**
- Static allocation
- Header binding

Explanation: Function overriding happens when a derived class redefines a base class virtual function. The correct implementation is determined at runtime based on the actual object, which is the definition of **Runtime polymorphism**.

Computer Architecture

41. Which architecture stores instructions and data in the same memory?

- Harvard architecture
- ✓ **Von Neumann architecture**
- Only RISC
- Only CISC

Explanation: The **Von Neumann architecture** uses a shared memory space and a single bus structure to hold both program instructions and data, which can lead to a performance limit known as the Von Neumann bottleneck

42. Which architecture separates instruction memory and data memory?

von Neumann architecture

✓ **Harvard architecture**

- x86 architecture only
- Stack architecture only

EXPLANATION: The **Harvard architecture** physically separates memory storage and signal pathways for instructions and data, allowing the CPU to read an instruction and access data simultaneously.

43. Heap memory is managed mainly by

- CPU registers
- ✓ **Runtime system and operating system support**
- Monitor
- BIOS only

EXPLANATION:

Heap allocations require the application's **runtime engine system** and the underlying **operating system** kernel to collaborate to map virtual memory pages, track free spaces, and handle process expansion blocks.

44. Which STL container performs dynamic memory allocation automatically?

- ✓ **vector**

- array only
- int
- char

EXPLANATION:

`std::vector` acts as a dynamic array wrapper. As elements are added, if the pre-allocated capacity is exceeded, it handles re-allocating memory and moving elements under the hood automatically.

45. Smart pointers were introduced to help solve

- Stack overflow only
- ✓ **Memory leak and ownership issues**
- Syntax errors
- Compilation speed

EXPLANATION:

Smart pointers implement the RAII (Resource Acquisition Is Initialization) pattern in C++. They manage raw pointer scope lifetimes to eliminate **memory leaks** and explicitly clarify resource ownership boundaries.

46. Which smart pointer allows only one owner?

- `shared_ptr`
- ✓ **`unique_ptr`**
- `weak_ptr`
- raw pointer

EXPLANATION:

`std::unique_ptr` maintains exclusive ownership of a heap resource. It cannot be copied or shared, ensuring that exactly one smart pointer tracks and manages that specific memory address.

47. Which smart pointer allows multiple owners?

- `unique_ptr`
- ✓ **`shared_ptr`**
- `weak_ptr`
- `nullptr`

EXPLANATION:

`std::shared_ptr` implements reference counting to allow multiple owners to track the same shared heap resource. The underlying resource is destroyed only when the last active reference is released.

48. Which smart pointer cannot be copied?

- shared_ptr
- ✓ **unique_ptr**
- weak_ptr
- auto_ptr only

EXPLANATION:

To maintain strict single ownership, std::unique_ptr explicitly disables its copy constructor and copy assignment operator. It can only be moved using std::move().

49. Which smart pointer helps observe shared_ptr without owning it?

- unique_ptr
- ✓ **weak_ptr**
- raw int pointer
- void pointer

EXPLANATION:

std::weak_ptr holds a non-owning reference to an object managed by std::shared_ptr. It prevents reference cycles (circular dependencies) that would otherwise cause memory leaks.

50. Which one is NOT a disadvantage of Dynamic Memory Allocation?

- Memory leak risk
- Fragmentation
- ✓ **Flexible memory usage**
- Slower than stack allocation

EXPLANATION:

Flexible memory usage is the primary benefit of dynamic memory allocation, allowing programs to request memory based on dynamic runtime needs rather than static compile-time constants.